PYBAQ · HERRAMIENTAS PARA MEETUPS

Cómo se construyó `raffle-cli`

Una guía para principiantes sobre el código, los diagramas y las decisiones técnicas detrás del proyecto

> **Versión cubierta:** 0.2.0 · **Python:** 3.10+ · **Código:** 1109 líneas en 9 módulos, 43 pruebas
>
> **Nota honesta:** el primer prototipo lo escribió Andoresu; esta versión (reestructuración, correcciones, persistencia, fuegos artificiales, la máquina tragamonedas en Textual, pruebas y documentación) se escribió junto con Claude, un asistente de IA. Esta guía es la forma de devolverte el aprendizaje: cada línea importante está explicada, cada decisión tiene una razón, y los ejercicios del final te piden cambiar el programa tú mismo.

Cómo usar esta guía. Lee un capítulo y luego abre el archivo del que habla y busca el código. Ejecuta el programa. Cambia un número, rompe algo a propósito y lee el error. Escribir código tú mismo enseña mucho más que leerlo. Los fragmentos de código se muestran tal como están en el repositorio (nombres y comentarios en inglés).

Contenido

1 Qué construimos

2 El panorama general

3 Punto de partida: la revisión

4 Fundamentos de Python que aparecen en este código

5 El motor del sorteo

6 Archivos, validación y reanudación

7 Aleatoriedad, justicia y --seed

8 La interfaz clásica de terminal

9 Los fuegos artificiales

10 La interfaz Textual

11 La máquina tragamonedas

12 Pruebas (tests)

13 Decisiones técnicas en una tabla

14 Lo que salió mal (y lo que enseñó)

15 Ejercicios

16 Glosario y cómo ejecutar todo

1 Qué construimos

`raffle-cli` hace un sorteo en la terminal para regalos en meetups: está pensado para ser *visible*, de modo que todos puedan ver que el sorteo es justo. Le das tres archivos (personas, premios y una configuración) y sortea los ganadores, un premio a la vez.

Puedes...	Con
Crear los archivos iniciales	`raffle-cli setup`
Hacer el sorteo clásico (Enter para revelar cada ganador)	`raffle-cli`
Usarlo como una tragamonedas a pantalla completa, con soporte de mouse	`raffle-cli --tui`
Ver solo el cierre con fuegos artificiales	`raffle-cli fireworks`
Hacer un sorteo reproducible y auditable	`--seed 7` (siempre se muestra una semilla en los resultados)
Elegir el orden de los premios, saltar las pausas, ejecutar sin intervención	`--order`, `--fast`, `--yes`
Recuperarte de una caída	los ganadores se guardan en el momento de revelarlos; vuelve a ejecutar y continúa

Los tres archivos

config.yaml

```yaml
participants_file: participants.csv
prizes_file: prizes.csv
winners_file: winners.csv
recreate_from_winners: true
```

participants.csv

```csv
name,lastname
Ada,Lovelace
Grace,Hopper
```

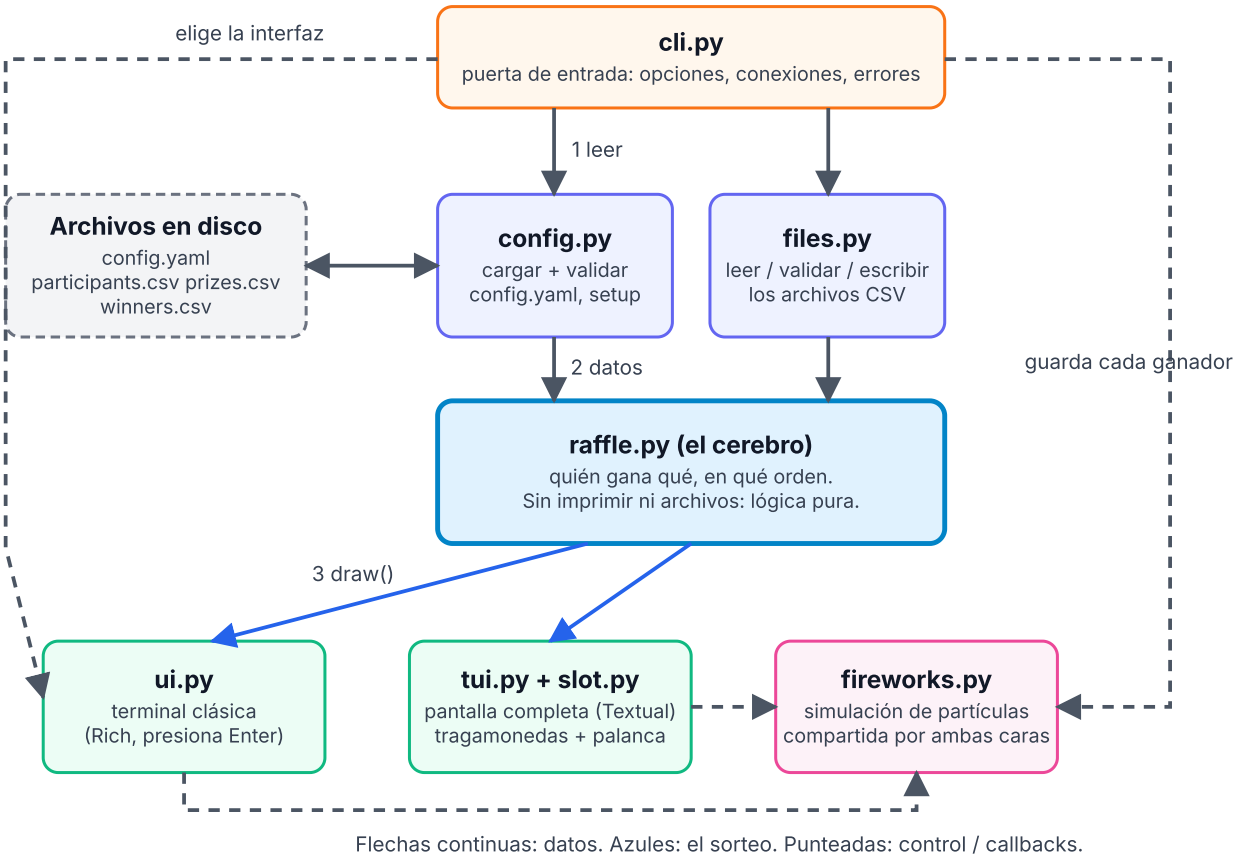
prizes.csv

```csv
item,qty
Mechanical keyboard,1
Python mug,2
```

El programa escribe un cuarto archivo, `winners.csv`, con las columnas `name,lastname,item`.

2 El panorama general

El proyecto original era un solo archivo con todo mezclado: leer archivos, sortear y mostrar en pantalla. El cambio más importante fue **separar esos trabajos**. Los programadores lo llaman *separación de responsabilidades*: cada pieza hace una sola cosa, y puedes cambiar o probar una pieza sin tocar las demás.

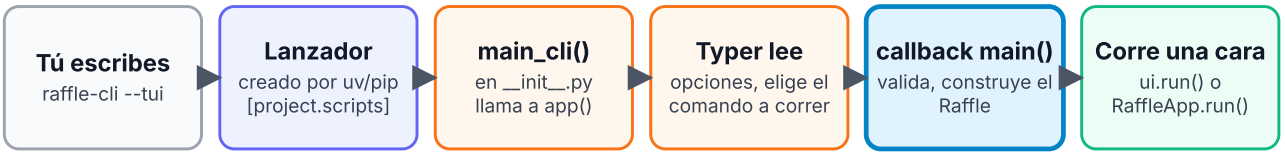


Archivo	Trabajo	Líneas	Ideas de Python que verás
`raffle.py`	El cerebro: quién gana qué. Sin imprimir, sin archivos.	131	dataclasses, anotaciones de tipo, tuplas como llave de orden, `random.Random`
`files.py`	Leer, validar y escribir los archivos CSV	145	`csv`, `pathlib`, excepciones propias, codificaciones
`config.py`	Cargar `config.yaml`, crear los archivos iniciales	77	modelos de Pydantic, YAML, clausuras (closures)
`cli.py`	La puerta de entrada: opciones, conexiones, errores amigables	143	Typer, `Annotated`, callbacks, códigos de salida
`ui.py`	La interfaz clásica	99	paneles y tablas de Rich, callbacks
`fireworks.py`	Simulación de partículas + dibujo	197	clases, física sencilla, `rich.live.Live`
`tui.py`	Aplicación Textual a pantalla completa	152	widgets, CSS, atajos de teclado, temporizadores, mensajes

`slot.py`	El widget de la tragamonedas	153	widget propio, animación con temporizadores, eventos de mouse

Qué pasa cuando escribes `raffle-cli`

Un comando como `raffle-cli` no es magia: la herramienta de empaquetado creó un pequeño script lanzador que importa tu función y la llama.



pyproject.toml: [project.scripts] raffle-cli = "raffle_cli:main_cli"

pyproject.toml (extracto)

```
[project.scripts]
raffle-cli = "raffle_cli:main_cli"

[project.optional-dependencies]
tui = ["textual>=0.86"]
```

3 Punto de partida: la revisión

El primer paso *no* fue escribir código: fue leer el prototipo y ejecutarlo para confirmar cada sospecha. Los errores que solo sospechas son opiniones; los que reproduces son hechos. Estos son los problemas reales que se encontraron y dónde se resuelve cada uno.

Problema encontrado (reproducido ejecutando el programa)	Solución	Capítulo
`raffle-cli setup` ejecutaba primero un sorteo completo (el callback principal corre antes que los subcomandos)	comprobación de `ctx.invoked_subcommand`	4
Los ganadores nunca se guardaban; `recreate_from_winners` no hacía nada	agregar cada ganador; lógica real de reanudación	6
Bucle infinito cuando hay más premios que participantes	sortear por rondas: todos ganan una vez antes de que alguien gane dos	5
`UnboundLocalError` con una configuración errónea	la validación lanza errores claros y sale con código 1	6
Los CSV vacíos o solo con encabezado se aceptaban y luego fallaban	la validación reporta cada problema con número de línea	6
Se usaba `eval()` para escoger constantes; nombres con `[` dañaban la salida	sin `eval` ; `rich.markup.escape`	4, 8
Dependencia `keyboard` sin usar (necesita root en Linux); `rich` usado pero no declarado	se arregló `pyproject.toml`	4
Un módulo de 485 líneas, sin pruebas, sin CI, sin LICENSE, README desactualizado	módulos, pruebas, CI, documentación	2, 12

> **Un hábito que vale la pena copiar**
> Antes de cambiar código, escribe qué está mal y demuéstralo. La mitad de los “errores” que sospechas resultan estar bien, y la otra mitad suele ser peor de lo que pensabas.

4 Fundamentos de Python que aparecen en este código

4.1 Empaquetado: de una carpeta a un comando

El código vive en `src/raffle_cli/`. Es el "diseño src": como el paquete no está en el nivel superior, tus pruebas solo pueden importarlo después de instalarlo. Eso evita un error clásico (código que funciona en tu computador solo porque estás parado en la carpeta correcta).

uv es la herramienta que crea un entorno virtual (un conjunto privado de paquetes instalados para un proyecto), instala lo que pide `pyproject.toml` y guarda las versiones exactas en `uv.lock` para que todos obtengan las mismas. `uv run pytest` ejecuta un comando dentro de ese entorno.

> **Decisión técnica: la interfaz Textual es un extra opcional**
>
> **Alternativa:** hacer de Textual una dependencia normal. **Por qué no:** la mayoría solo quiere la interfaz clásica y Textual es una dependencia grande. Con un extra (`raffle-cli[tui]`) se instala solo si se pide. El código lo importa de forma diferida, dentro de la rama `if tui:` , y muestra un mensaje útil si falta. **Costo:** una cosa más que los usuarios deben recordar (`uvx --with textual`).

4.2 Anotaciones de tipo y dataclasses

Una **dataclass** es una clase cuyo trabajo principal es guardar datos. Python escribe por ti `__init__` , `__repr__` y `__eq__` . `frozen=True` hace las instancias de solo lectura, lo que además permite compararlas y usarlas como llaves de diccionario. El texto después de los dos puntos (`str` , `int`) es una *anotación de tipo*: Python no la impone, pero tu editor y quienes leen el código sí la aprovechan.

src/raffle_cli/raffle.py · Participant

```python
@dataclass(frozen=True)
class Participant:
    name: str
    lastname: str

    @property
    def full_name(self) -> str:
        return f"{self.name} {self.lastname}"
```

src/raffle_cli/raffle.py · Prize

```python
@dataclass(frozen=True)
class Prize:
    item: str
    qty: int
```

src/raffle_cli/raffle.py · Winner

```python
@dataclass(frozen=True)
class Winner:
    name: str
    lastname: str
    item: str

    @property
    def full_name(self) -> str:
        return f"{self.name} {self.lastname}"
```

> **Decisión técnica: dataclasses pequeñas e inmutables en vez de diccionarios**
>
> El prototipo pasaba de un lado a otro diccionarios `{"name": ..., "lastname": ...}`. Un error de dedo como `row["lasname"]` solo falla cuando esa línea se ejecuta. Con una dataclass, `p.lasname` lo marca tu editor de inmediato, las pruebas pueden comparar objetos completos con `==` y `frozen` garantiza que nadie modifique un premio por accidente. **Alternativa:** modelos de Pydantic (más funciones, más peso; usamos Pydantic solo donde vale la pena: validar `config.yaml`).

La línea `from __future__ import annotations` al inicio de cada módulo vuelve las anotaciones de tipo cadenas diferidas, así que la sintaxis moderna como `list[str] | None` es segura en Python 3.10.

4.3 pathlib, csv y codificaciones

`pathlib.Path` trata las rutas como objetos: `base / "winners.csv"` une rutas correctamente en Windows y Linux. El módulo `csv` se encarga de las comillas (un premio llamado `Mug, large` sigue siendo una sola celda). Un detalle del mundo real: Excel guarda los CSV con una *marca de orden de bytes* invisible al inicio. Abrimos los archivos con `encoding="utf-8-sig"`, que la elimina en silencio; de lo contrario la primera columna se llamaría `\ufeffname` y la validación fallaría por una razón misteriosa.

4.4 Excepciones que llevan información

src/raffle_cli/files.py · DataFileError

```python
class DataFileError(Exception):
    """A work file is missing or invalid; `problems` lists everything wrong with it."""

    def __init__(self, path: Path, problems: list[str]) -> None:
        self.path = path
        self.problems = problems
        super().__init__(f"{path.name}: " + "; ".join(problems))
```

Una excepción es solo una clase. Esta guarda *todos* los problemas encontrados en un archivo, no solo el primero. Quien arregla una hoja de cálculo quiere ver todas las filas incorrectas de una vez. La línea de comandos la captura e imprime una lista, así los usuarios nunca ven un traceback por sus propios errores de digitación.

4.5 Pydantic para el archivo de configuración

src/raffle_cli/config.py · RaffleConfig

```python
class RaffleConfig(BaseModel):
    participants_file: Path = Path("participants.csv")
    prizes_file: Path = Path("prizes.csv")
    winners_file: Path = Path("winners.csv")
    # True: resume from whatever is already in winners_file. False: start fresh each run.
    recreate_from_winners: bool = True

    def resolved(self, base: Path) -> "RaffleConfig":
        """Make relative file paths relative to the config file's folder."""
        return RaffleConfig(
            participants_file=base / self.participants_file,
            prizes_file=base / self.prizes_file,
            winners_file=base / self.winners_file,
            recreate_from_winners=self.recreate_from_winners,
        )
```

Pydantic revisa la forma de los datos usando las anotaciones de tipo. Si alguien escribe `recreate_from_winners: maybe`, Pydantic lanza `ValidationError`, que `load_config` convierte en un solo mensaje legible. Las rutas se resuelven relativas a la carpeta del archivo de configuración (`resolved()`), así que `raffle-cli -c event/config.yaml` funciona desde cualquier lugar.

4.6 Typer: convertir funciones en comandos

Typer lee los parámetros de una función y construye la línea de comandos a partir de ellos. `@app.command()` agrega un subcomando (`setup`, `fireworks`); `@app.callback(invoke_without_command=True)` define el código que corre para `raffle-cli` mismo y guarda las opciones globales.

src/raffle_cli/cli.py · el callback principal (inicio)

```python
@app.callback(invoke_without_command=True)
def main(
    ctx: typer.Context,
    config: ConfigOption = None,
    seed: Annotated[Optional[int], typer.Option(help="Seed the draw so it can be reproduced. Shown in the results.")] = None,
    order: Annotated[str, typer.Option(help=f"Order prizes are raffled: {', '.join(ORDERS)}.")] = "high-low",
    fast: Annotated[bool, typer.Option("--fast", help="Skip the suspense delays.")] = False,
    yes: Annotated[bool, typer.Option("--yes", "-y", help="Never wait for Enter (non-interactive).")] = False,
    fireworks_flag: Annotated[bool, typer.Option("--fireworks/--no-fireworks", help="Finish with fireworks.")] = True,
    tui: Annotated[bool, typer.Option("--tui", help="Use the full-screen Textual interface.")] = False,
    version: Annotated[bool, typer.Option("--version", callback=version_callback, is_eager=True, help="Show the version.")] = False,
) -> None:
    """Run the raffle (or a subcommand)."""
    ctx.obj = {"config": config}
```

… y lo que hace después

```python
if ctx.invoked_subcommand is not None:
    return
if order not in ORDERS:
    raise fail(f"--order must be one of: {', '.join(ORDERS)}.")
seed = secrets.randbits(32) if seed is None else seed
raffle, cfg = build_raffle(config or DEFAULT_CONFIG, seed, order, interactive=not yes)
```

Decisión técnica: una guarda en el callback en vez de un comando “run” aparte

El callback del prototipo ejecutaba el sorteo *siempre*, incluso con `raffle-cli setup`. La corrección son dos líneas: si Typer indica que se pidió un subcomando (`ctx.invoked_subcommand`), se retorna de inmediato. **Alternativa:** un `raffle-cli run` dedicado; más limpio en teoría, pero rompería el simple y fácil de recordar `raffle-cli` del README original. **Costo:** opciones como `--seed` viven en el callback, así que van *antes* del nombre de un subcomando.

`Annotated[bool, typer.Option("--fast", help="…")] = False` se lee así: “un bool, expuesto como la opción `--fast` con este texto de ayuda, por defecto False”. Los errores pasan por una sola función, `fail()`, que imprime en rojo y devuelve `typer.Exit(1)`; el código de salida distinto de cero permite que los scripts detecten el fallo.

4.7 Rich: color sin dolor

Rich imprime texto con estilo usando un pequeño lenguaje de marcado: `[bold green]hola[/]`. El problema: los datos del usuario pueden contener corchetes. Un participante llamado `[bold]Eve` se interpretaría como marcado. Todos los lugares que imprimen nombres usan `rich.markup.escape()`.

5 El motor del sorteo

`raffle.py` es el corazón. No importa ninguna biblioteca de interfaz, no imprime nada y no toca archivos. Por eso es fácil de probar y por eso *dos* interfaces distintas (clásica y Textual) pueden compartirlo sin duplicar una sola línea de la lógica del sorteo.

5.1 El modelo de datos: una pila de unidades de premio

src/raffle_cli/raffle.py · Raffle.__init__

```python
def __init__(
    self,
    participants: Sequence[Participant],
    prizes: Sequence[Prize],
    *,
    previous_winners: Iterable[Winner] = (),
    rng: random.Random | None = None,
    order: str = "high-low",
) -> None:
    if not participants:
        raise ValueError("A raffle needs at least one participant.")
    self.rng = rng or random.Random()
    self.participants = list(participants)
    self.winners: list[Winner] = []
    self._wins = [0] * len(self.participants)
    self._held: list[set[str]] = [set() for _ in self.participants]
    self._units: list[str] = [
        p.item for p in order_prizes(prizes, order, self.rng) for _ in range(p.qty)
    ]
    for winner in previous_winners:
        self._restore(winner)
```

En vez de llevar la cuenta de “Mug: quedan 2”, el constructor expande cada premio en *una entrada por cada objeto físico*: `["Mug", "Mug", "Pen", ...]`. Entregar un premio es simplemente `pop(0)`. Contar los premios restantes es contar la lista. Menos casos especiales significa menos errores.

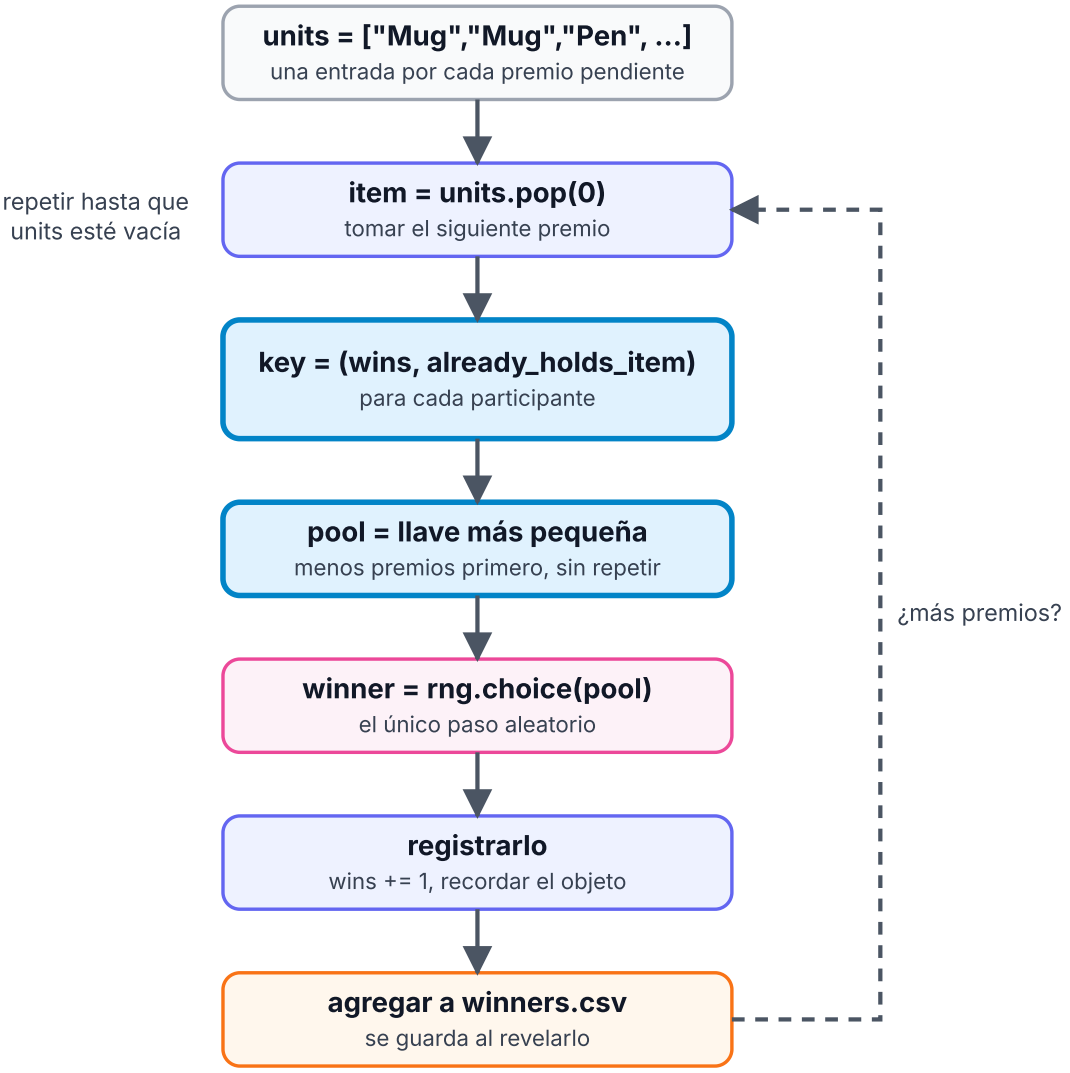
src/raffle_cli/raffle.py · order_prizes

```python
def order_prizes(prizes: Sequence[Prize], order: str, rng: random.Random) -> list[Prize]:
    """Return a new list of prizes sorted by `order` ('high-low', 'low-high', 'shuffle' or 'file')."""
    ordered = list(prizes)
    if order == "shuffle":
        rng.shuffle(ordered)
    elif order == "low-high":
        ordered.sort(key=lambda p: p.qty)
    elif order == "high-low":
        ordered.sort(key=lambda p: p.qty, reverse=True)
    elif order != "file":
        raise ValueError(f"Invalid order {order!r}. Choose from {', '.join(ORDERS)}.")
    return ordered
```

Decisión técnica: nunca modificar los datos de quien llama

El prototipo restaba `prize["qty"]` en el mismo lugar, así que la misma lista no se podía reutilizar (para una segunda ejecución, una prueba o volver a dibujar). `order_prizes` devuelve una lista *nueva*; `Prize` es inmutable; el motor guarda sus propias `_units`. Una prueba (`test_prizes_are_not_mutated`) lo asegura.

5.2 Cómo se elige un ganador



src/raffle_cli/raffle.py · Raffle.draw

```
def draw(self) -> Winner:
    if self.done:
        raise RuntimeError("All prizes have been awarded.")
    item = self._units.pop(0)
    keys = [(self._wins[i], item in self._held[i]) for i in range(len(self.participants))]
    best = min(keys)
    pool = [i for i, key in enumerate(keys) if key == best]
    return self._record(self.rng.choice(pool), item)
```

La idea central cabe en una línea: `keys = [(wins, holds_item) ...]`. Python compara las tuplas elemento por elemento, así que `(0, False)` va antes que `(1, False)`, que va antes que `(1, True)`. `min(keys)` da la llave de quien "más lo merece"; `pool` son todos los que la comparten; `rng.choice(pool)` es el único paso aleatorio. Las reglas salen de la tupla:

- **Nadie gana dos veces antes de que todos hayan ganado una** (primer elemento: menos victorias).
- **En rondas posteriores, evitar darle a alguien un objeto que ya tiene** (segundo elemento), salvo que no haya alternativa.

Un ejemplo resuelto

Tres personas (Ana, Ben, Cy), premios Mug×2, Pen×2, Sticker×1. El orden `high-low` (estable) da `units = [Mug, Mug, Pen, Pen, Sticker]`. Una ejecución posible:

#	Premio	Grupo (llave más pequeña)	Ganador	Victorias después (Ana, Ben, Cy)
1	Mug	Ana, Ben, Cy: todos (0, no)	Ben	0, 1, 0
2	Mug	Ana, Cy: aún con 0 victorias	Cy	0, 1, 1
3	Pen	Ana: la única con 0 victorias	Ana	1, 1, 1
4	Pen	Ben, Cy: Ana es (1, **tiene Pen**), así que va detrás de ellos	Cy	1, 1, 2
5	Sticker	Ana, Ben: menos victorias (1)	Ben	1, 2, 2

Cinco premios, tres personas: nadie recibió un segundo premio hasta el sorteo 4, porque solo entonces todos habían ganado una vez. Esa es exactamente la regla "si hay más premios que participantes, la gente recibe más de uno".

Decisión técnica: rondas en vez de "elegir a cualquiera y luego cambiar"

El prototipo escogía una persona al azar con reemplazo; si ya tenía un premio preguntaba "¿cambiar?". Eso volvía el sorteo interactivo de una forma confusa, y se quedaba en un bucle infinito cuando había más premios que personas, porque cambiar nunca reducía la pila. **Nuestra regla:** sortear entre quienes tienen menos victorias. Es fácil de explicar a una audiencia, nunca entra en bucle infinito y es demostrablemente justa (las pruebas verifican que con 10 premios y 5 personas cada una recibe exactamente dos). **Costo:** eliminamos la pregunta de cambio. Podría volver como opción.

Decisión técnica: los participantes se identifican por posición, no por nombre

Dos personas pueden tener el mismo nombre. Si los ganadores se buscaran por `(name, lastname)`, se fundirían en uno. El motor guarda `_wins` y `_held` como listas indexadas por la posición del participante, así que los duplicados son dos oportunidades distintas. El lector de CSV solo imprime una *advertencia* por duplicados.

5.3 Reconstruir el estado: reanudar

src/raffle_cli/raffle.py · Raffle._restore

```python
def _restore(self, winner: Winner) -> None:
    try:
        self._units.remove(winner.item)
    except ValueError:
        raise ValueError(
            f"Winners file lists '{winner.item}' for {winner.full_name}, "
            "but there are no such prizes left in the prizes file."
        ) from None
    candidates = [
        i for i, p in enumerate(self.participants)
        if p.name == winner.name and p.lastname == winner.lastname
    ]
    if not candidates:
        raise ValueError(f"Winner {winner.full_name} is not in the participants file.")
    i = min(candidates, key=lambda c: self._wins[c])
    self._record(i, winner.item)
```

Al reanudar, cada ganador guardado se reproduce: se quita ese premio de la pila y se cuenta la victoria. Si el archivo guardado contradice el archivo de premios (un objeto que ya no existe, una persona que ya no está) lanzamos un `ValueError` claro en vez de adivinar.

6 Archivos, validación y reanudación

6.1 Leer un CSV a la defensiva

La regla para todo lo que escribe un humano: **asume que está mal y explica exactamente cómo**. Los tres lectores comparten una función auxiliar que revisa el encabezado y devuelve las filas con sus números de línea.

src/raffle_cli/files.py · _read_rows

```python
def _read_rows(path: Path, columns: tuple[str, ...]) -> list[tuple[int, dict[str, str]]]:
    """Read `path`, check its header and return (line_number, stripped_row) pairs."""
    if not path.exists():
        raise DataFileError(path, [f"file not found ({path}). Run `raffle-cli setup` to create it."])
    with open(path, newline="", encoding="utf-8-sig") as fh:
        reader = csv.DictReader(fh)
        header = [h.strip().lower() for h in (reader.fieldnames or [])]
        if not header:
            raise DataFileError(path, [f"missing header row, expected: {','.join(columns)}"])
        if set(header) != set(columns) or len(header) != len(columns):
            raise DataFileError(
                path, [f"expected columns {','.join(columns)} but found {','.join(header)}"]
            )
        reader.fieldnames = header
        rows = []
        for row in reader:
            clean = {k: (v or "").strip() for k, v in row.items() if k is not None}
            if any(clean.values()):
                rows.append((reader.line_num, clean))
    return rows
```

src/raffle_cli/files.py · read_prizes

```python
def read_prizes(path: Path) -> list[Prize]:
    problems: list[str] = []
    prizes: list[Prize] = []
    for line, row in _read_rows(path, PRIZE_COLUMNS):
        if not row["item"]:
            problems.append(f"line {line}: item is required")
            continue
        try:
            qty = int(row["qty"])
        except ValueError:
            problems.append(f"line {line}: qty must be a whole number, got '{row['qty']}'")
            continue
        if qty < 1:
            problems.append(f"line {line}: qty must be at least 1, got {qty}")
            continue
        prizes.append(Prize(row["item"], qty))
    if not problems and not prizes:
        problems.append("no prizes found")
    if problems:
        raise DataFileError(path, problems)
    return prizes
```

Decisión técnica: reunir todos los problemas y nunca "reparar" los archivos del usuario

Dos decisiones aquí.

1. Reportar todos los problemas a la vez. Fallar en la primera fila mala significa diez ciclos de editar y ejecutar para diez errores. El lector junta una lista y lanza el error una sola vez.

2. Sin arreglos automáticos. El prototipo ofrecía "¿quieres poner el contenido por defecto?", lo que sobrescribiría una lista real de participantes si presionabas `y` por reflejo. Ahora un archivo dañado produce un mensaje de error y se deja intacto; `setup` solo escribe archivos que no existen (o cuando pasas `--force`).

Detalles menores: los encabezados se pasan a minúsculas y se recortan, las celdas se recortan, las líneas en blanco se ignoran y `qty` debe ser un entero ≥ 1.

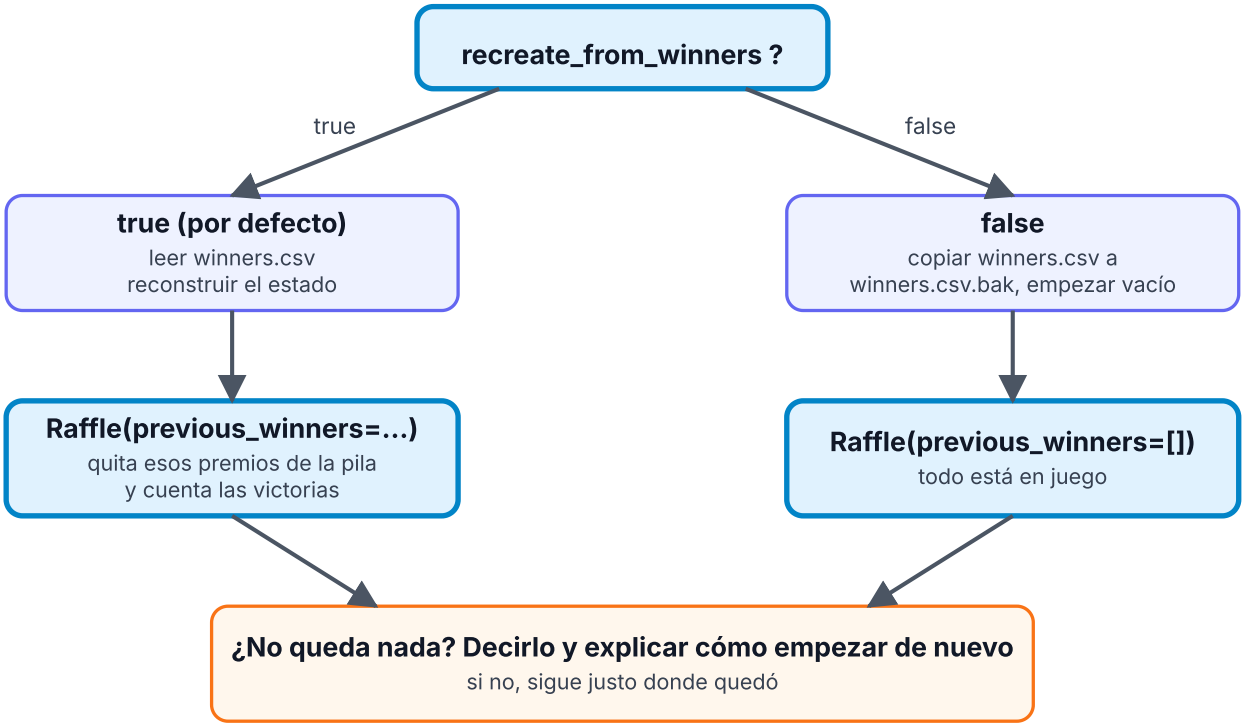
6.2 Guardar cada ganador de inmediato

`src/raffle_cli/files.py · append_winner`

```python
def append_winner(path: Path, winner: Winner) -> None:
    """Persist one winner immediately, so a crash mid-event loses nothing."""
    if not path.exists():
        reset_winners(path)
    with open(path, "a", newline="", encoding="utf-8") as fh:
        csv.writer(fh).writerow((winner.name, winner.lastname, winner.item))
```

Si el portátil se apaga en medio de un meetup, no deberías perder los ganadores ya anunciados. Por eso un ganador se agrega a `winners.csv` en el momento en que se revela, no al final. Agregar una sola fila además es barato y seguro.

6.3 Empezar de cero o reanudar



Decisión técnica: el ganador se decide primero y se guarda al revelarlo

`raffle.draw()` escoge al ganador *antes* del suspenso, luego la interfaz reproduce la animación, y solo cuando se revela el nombre llamamos al callback `on_winner` que escribe el CSV. **Por qué:** la interfaz puede mostrar lo que quiera mientras el resultado ya está fijo (los rodillos de la tragamonedas simplemente aterrizan en él), y el archivo nunca contiene un ganador que la audiencia no haya visto aún. **Costo:** si sales durante un giro, ese premio no queda registrado y se sorteará de nuevo al reanudar (posiblemente otra persona): es el comportamiento correcto para un evento en vivo.

Decisión técnica: los resultados anteriores se respaldan, no se borran

Con `recreate_from_winners: false` el programa empieza vacío, pero antes copia un `winners.csv` no vacío a `winners.csv.bak` (ver `reset_winners`). Borrar resultados en silencio es el tipo de error que solo cometes una vez, frente a una audiencia.

7 Aleatoriedad, justicia y --seed

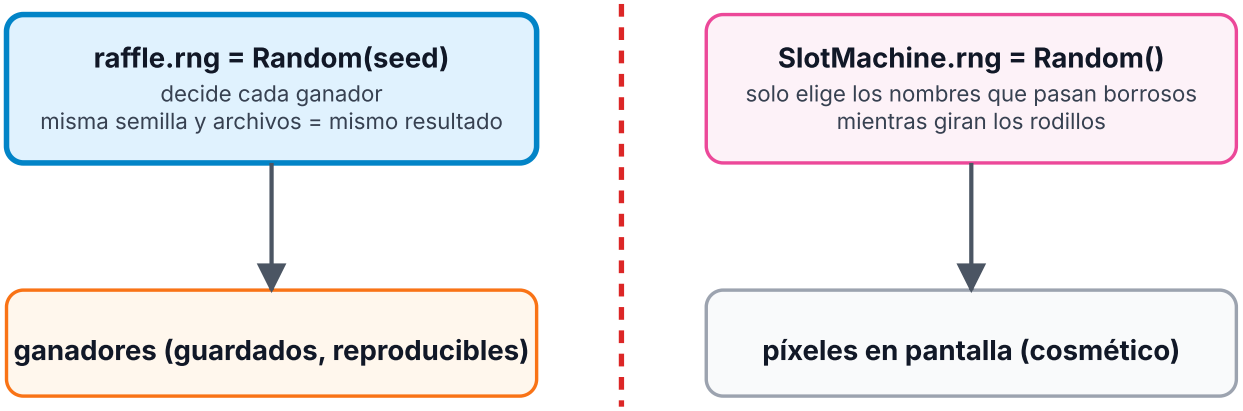
Los computadores no producen números verdaderamente aleatorios; `random.Random(seed)` es un *generador pseudoaleatorio*: dale la misma semilla y produce la misma secuencia siempre. Suena como un defecto, pero para un sorteo transparente es una ventaja.

- Si no se da una semilla, se genera una con `secrets.randbits(32)` y se **muestra en los resultados**.
- Cualquiera con los mismos archivos y la misma `--seed` puede reproducir exactamente los mismos ganadores.

> **Decisión técnica: mostrar la semilla en vez de ocultar el azar**
>
> **Alternativa:** `random.SystemRandom()`, que toma la entropía del sistema operativo: impredecible, pero nadie puede verificar jamás un resultado. Para regalar stickers, la *auditabilidad* pesa más que el secreto. Una semilla de 32 bits no es criptográficamente fuerte y eso está bien: es un recibo, no un secreto. (Si los premios tuvieran valor en dinero, querrías comprometerte públicamente con la semilla antes del sorteo.)

7.1 Un error que encontramos pensando

La primera versión de Textual hacía el efecto de "nombres que pasan borrosos" llamando a `raffle.rng.choice(...)`. Cada llamada consume un número del generador, así que la animación del giro cambiaba *quién ganaba el siguiente premio*: la misma semilla daba ganadores distintos en la interfaz clásica y en la de Textual, y con `--fast` (menos pasos de animación) frente a velocidad normal. La solución: la tragamonedas tiene su propio generador aparte.



nunca se comparten: la animación no puede cambiar quién gana

> **Regla práctica**
>
> Todo lo que deba ser reproducible tiene su propio generador aleatorio, y nada cosmético puede tocarlo. Una prueba (`test_tui_animation_leaves_the_seeded_draw_untouched`) ahora lo vigila.

8 La interfaz clásica de terminal

`ui.py` es la primera "cara". Recibe un `Raffle` listo y una función `on_winner` y se ocupa solo de la presentación.

src/raffle_cli/ui.py · run

```python
def run(
    raffle: Raffle,
    console: Console,
    on_winner: Callable[[Winner], None],
    *,
    seed: int | None = None,
    fast: bool = False,
    auto: bool = False,
    show_fireworks: bool = True,
) -> None:
    """Run the raffle until every prize is awarded."""
    word_delay = 0.0 if fast else 0.3
    if not raffle.done:
        delayed(console, "Game starting in...", word_delay)
        delayed(console, "3 2 1 Go", 0 if fast else 1)
        time.sleep(0 if fast else 0.5)
    while not raffle.done:
        item = raffle.next_prize
        title(console, "[purple]ONGOING RAFFLE[/]", "raffle-cli", "Let's Raffle!")
        console.print(Panel(Columns([prizes_table(raffle), winners_table(raffle.winners)])))
        delayed(console, f"Next item to win is... {item}", word_delay)
        winner = raffle.draw()
        delayed(console, f"\tThe winner of {item} is.... ", word_delay)
        pause(console, "\tPress enter when ready.", auto)
        console.print(f"\t\t[bold green]{escape(winner.full_name)}[/]")
        on_winner(winner)
        if not raffle.done:
            pause(console, "Press enter to continue...", auto)

    show_results(console, raffle.winners, seed)
    if show_fireworks:
        pause(console, "\nPress enter for the finale...", auto)
        captions = [f"{w.full_name}  ·  {w.item}" for w in raffle.winners]
        fireworks.play(console, seconds=3 if fast else 10, captions=captions)
        show_results(console, raffle.winners, seed)
```

Fíjate en lo que *no* sabe: cómo se eligen los ganadores (`raffle.draw()`) ni dónde se guardan (`on_winner`). Pasar una función como argumento así se llama **callback**; mantiene la interfaz y el almacenamiento independientes.

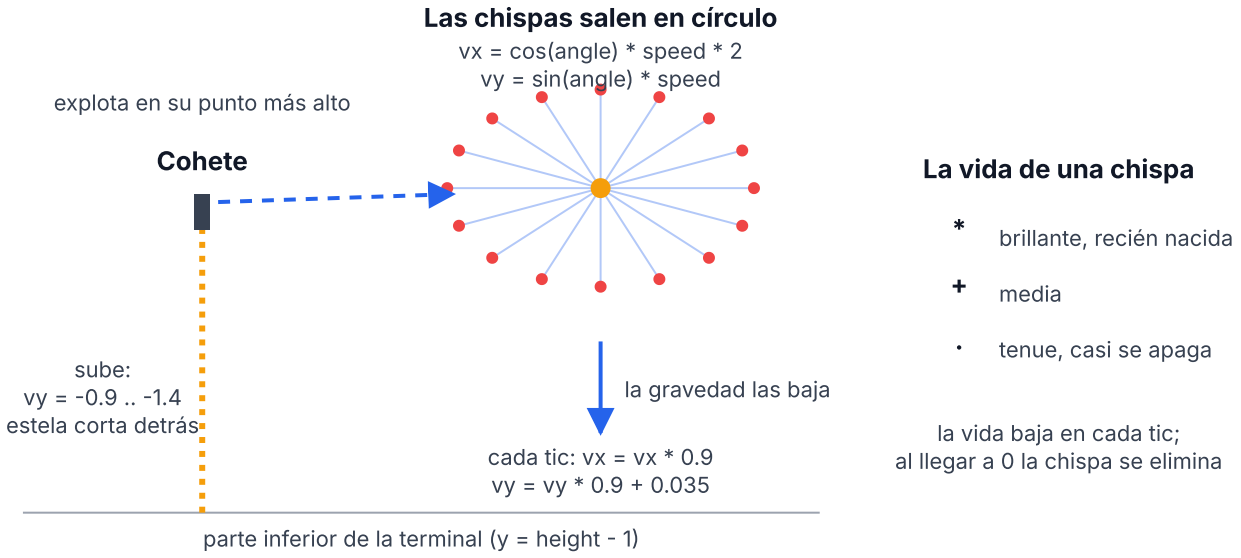
- `delayed()` imprime palabra por palabra con `time.sleep` para crear suspenso; `--fast` pone la pausa en cero.
- `pause()` espera Enter, pero captura `EOFError` para que el programa no falle cuando la entrada viene por una tubería (por ejemplo `echo | raffle-cli` en un script).
- El mismo objeto `Raffle` se muestra en tablas mediante `raffle.remaining` y `raffle.winners`.

src/raffle_cli/ui.py · pause

```python
def pause(console: Console, prompt: str, auto: bool) -> None:
    if auto:
        return
    try:
        console.input(f"[dim]{prompt}[/]")
    except EOFError:  # piped stdin: carry on instead of crashing
        pass
```

9 Los fuegos artificiales

Los fuegos artificiales son una primera *simulación* clásica: mantén una lista de cosas en movimiento, actualiza cada una un poquito por "tic", dibújalas todas y repite 20 veces por segundo. Hay dos tipos de cosas: **cohetes** (suben y luego explotan) y **chispas** (salen disparadas, frenan, caen y se apagan).



src/raffle_cli/fireworks.py · Fireworks.step

```python
def step(self) -> None:
    self.tick += 1
    if self.rng.random() < self.launch_rate:
        self._launch()
    for rocket in self.rockets[:]:
        rocket.y += rocket.vy
        self.sparks.append(Spark(rocket.x, rocket.y + 1, 0, 0, 5, "orange3"))
        if rocket.y <= rocket.apex:
            self.rockets.remove(rocket)
            self._explode(rocket)
    for s in self.sparks[:]:
        s.x += s.vx
        s.y += s.vy
        s.vx *= DRAG
        s.vy = s.vy * DRAG + GRAVITY
        s.life -= 1
        if s.life <= 0 or not (-1 <= s.x <= self.width and -1 <= s.y <= self.height):
            self.sparks.remove(s)
```

src/raffle_cli/fireworks.py · Fireworks._explode

```python
def _explode(self, rocket: Rocket) -> None:
    rng = self.rng
    count = rng.randint(30, 55)
    shape = rng.choice(["ring", "burst"])
    colors = [rocket.color] if rng.random() < 0.6 else [rocket.color, rng.choice(PALETTE)]
    ttl = rng.randint(14, 24)
    for i in range(count):
        angle = 2 * math.pi * i / count
        speed = 0.75 if shape == "ring" else rng.uniform(0.25, 0.8)
        self.sparks.append(
            Spark(
                rocket.x,
                rocket.y,
                math.cos(angle) * speed * ASPECT,
                math.sin(angle) * speed,
                ttl + rng.randint(-3, 3),
                rng.choice(colors),
            )
        )
```

Tres pedacitos de matemáticas hacen el trabajo:

- **Ángulo:** `2π · i / count` reparte las chispas de forma pareja alrededor de un círculo. Las explosiones “ring” usan todas la misma velocidad; las “burst” usan velocidades al azar.
- **Proporción de aspecto:** las celdas de la terminal son aproximadamente el doble de altas que de anchas, así que la velocidad horizontal se multiplica por `ASPECT = 2`. Sin eso, los círculos se ven como óvalos altos.
- **Fricción y gravedad:** en cada tic la velocidad se multiplica por 0,9 (resistencia del aire) y la gravedad suma 0,035 a la velocidad hacia abajo. Una chispa con velocidad 0,75 recorre unas `0,75 / (1 − 0,9) ≈ 7,5` filas antes de detenerse, que es justo lo adecuado para una terminal.

9.1 Dibujar un cuadro

`render()` crea una cuadrícula vacía de caracteres, “dibuja” en ella cada cohete y cada chispa (el carácter depende de cuánta vida le queda: `*`, `+`, `.`), superpone el título y el texto del ganador, y devuelve un solo `rich.text.Text`. Nota que `Fireworks` nunca imprime nada por sí mismo.

src/raffle_cli/fireworks.py · play

```python
def play(
    console: Console,
    *,
    seconds: float = 8.0,
    fps: int = 20,
    title: str = "CONGRATULATIONS",
    captions: list[str] | tuple[str, ...] = (),
    rng: random.Random | None = None,
) -> None:
    """Run the show full-screen. Ctrl+C skips it. Does nothing when not attached to a terminal."""
    if not console.is_terminal:
        return
    width, height = console.size
    show = Fireworks(width, height, rng, title=title, captions=captions)
    for _ in range(8):
        show._launch()
    try:
        with Live(console=console, screen=True, auto_refresh=False) as live:
            for _ in range(int(seconds * fps)):
                show.resize(*console.size)
                show.step()
```

```
            live.update(show.render(), refresh=True)
            time.sleep(1 / fps)
    except KeyboardInterrupt:
        pass
```

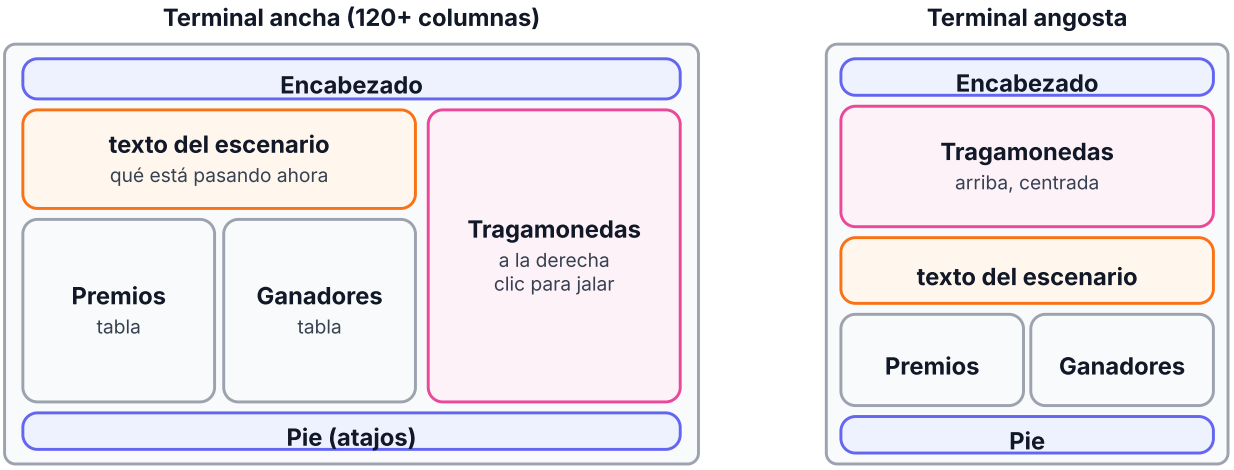
Decisión técnica: una simulación, dos formas de dibujarla

`Fireworks` solo produce un `Text`. `play()` lo muestra con `Live(screen=True)` de Rich (la "pantalla alterna" de la terminal, que se restaura después, como `vim` o `less`), y la interfaz Textual muestra el mismo objeto dentro de un widget `Static` refrescado por un temporizador. **Alternativas:** un programa con curses (no portable a Windows) o una biblioteca de fuegos artificiales (una dependencia para 200 líneas de código). **Costo:** implementamos la física nosotros mismos, lo que además es más divertido para aprender. No hace nada cuando no hay una terminal conectada (`console.is_terminal`), así que los scripts y las pruebas no se cuelgan.

10 La interfaz Textual

Textual es un framework para aplicaciones de terminal a pantalla completa con ideas tomadas de las páginas web: *widgets* (tablas, etiquetas, los tuyos), diseño descrito con *CSS*, *eventos* (teclado, mouse, cambio de tamaño), *temporizadores* y *mensajes* que los widgets se envían entre sí.

10.1 Un diseño que se adapta a la ventana



Una clase CSS en #main ("wide") cambia entre los dos diseños al redimensionar la ventana.

src/raffle_cli/tui.py · constantes y CSS

```
WIDE_AT = 120  # columns; at least this wide the slot machine moves to the right


class RaffleApp(App):
    TITLE = "raffle-cli"
    CSS = """
    #main { height: 1fr; }
    #machine-box { height: 8; width: 100%; align: center middle; }
    #main.wide #machine-box { dock: right; width: 52; height: 100%; }
    #stage { height: 6; content-align: center middle; text-align: center; border: round $accent; margin:
0 1; }
    #tables { height: 1fr; }
    #tables > Vertical { width: 1fr; padding: 0 1; }
    """
    BINDINGS = [
```

src/raffle_cli/tui.py · RaffleApp._layout

```
def _layout(self, width: int) -> None:
    self.query_one("#main").set_class(width >= WIDE_AT, "wide")
```

La tragamonedas está en un contenedor (`#machine-box`) que por defecto es una fila de ancho completo en la parte superior. Cuando la ventana tiene al menos 120 columnas de ancho, Python agrega la clase CSS `wide` a `#main`, y la regla `#main.wide #machine-box { dock: right; ... }` la mueve al borde derecho. Python decide *cuándo*; el CSS decide *cómo se ve*. El cambio de tamaño lo maneja `on_resize`, que vuelve a llamar a `_layout`.

10.2 Atajos de teclado y un error sorprendente

src/raffle_cli/tui.py · RaffleApp.action_draw

```
def action_draw(self) -> None:
    if self._busy or self.raffle.done:
        return
    self._busy = True
    item = self.raffle.next_prize
    winner = self.raffle.draw()  # decided now; the reels just land on it
    self._stage(f"Drawing [b yellow]{escape(item)}[/]\n\n[dim]Good luck![/]")

    def reveal() -> None:
        self.on_winner(winner)
        self._refresh()
        hint = "That was the last prize!" if self.raffle.done else "Press Enter or click the machine for
the next prize"
        self._stage(
            f"[b yellow]{escape(item)}[/] goes to\n\n[b green]🎉 {escape(winner.full_name)} 🎉
[/]\n\n[dim]{hint}[/]"
        )
        self._busy = False
        if self.raffle.done and self.show_fireworks:
            self.set_timer(0.5 if self.fast else 3.0, self.action_fireworks)

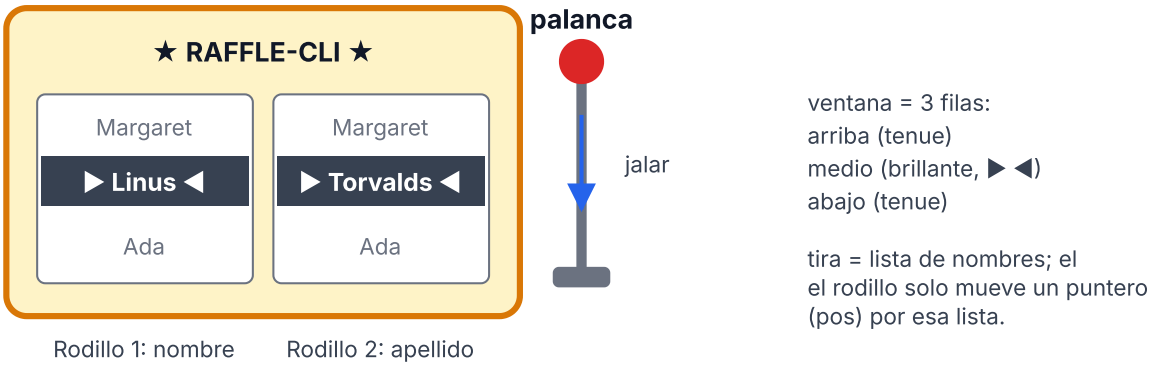
    self.query_one(SlotMachine).pull(winner, reveal)
```

`BINDINGS` asocia teclas con *acciones* (métodos llamados `action_...`). Nuestra primera versión no hacía nada al presionar Enter, porque el `DataTable` enfocado maneja Enter por su cuenta ("seleccionar esta fila") antes de que la aplicación lo vea. La solución es `priority=True` en el atajo, que deja ganar a la aplicación. La batería de pruebas lo detectó: presiona Enter y verifica que se sorteó un premio.

> **Decisión técnica: la aplicación le dice a los widgets qué hacer; los widgets envían mensajes de vuelta**
>
> Fíjate en que `action_draw` llama a `machine.pull(winner, reveal)` (la aplicación le ordena al widget), mientras que el widget le dice a la aplicación "me hicieron clic" *publicando un mensaje* (`Pulled`). La aplicación nunca se mete en las entrañas del widget y el widget no sabe nada de sorteos aparte de un ganador en el que aterrizar. Es la misma idea de callback del capítulo 8.

11 La máquina tragamonedas

La tragamonedas es un **widget propio**: una clase que hereda del `Widget` de Textual e implementa `render()`, devolviendo lo que se debe dibujar. Todo lo que ves es texto: caracteres de dibujo de cajas, unos cuantos colores, y la palanca hecha de `●`, `┃` y `┻`.



Un **rodillo** es solo una lista de nombres (la tira) y un número (`pos`) que dice cuál está en la fila del medio. “Girar” significa aumentar `pos` y volver a dibujar. Para que el rodillo aterrice en el ganador, la tira se construye *al revés*: nombres al azar, con el ganador colocado en el índice `steps` , y el rodillo se detiene cuando `pos` lo alcanza. Los nombres justo encima y debajo del ganador se vuelven a sortear para que nunca veas el nombre del ganador dos veces.

src/raffle_cli/slot.py · SlotMachine._start_reels

```python
def _start_reels(self, winner: Winner) -> None:
    self._pending = len(self.reels)
    for i, part in enumerate((winner.name, winner.lastname)):
        steps = (3 if self.fast else 18) + i * (2 if self.fast else 12)  # the right reel spins longer
        strip = self._random_strip(i, steps + 2)
        strip[steps] = part
        for neighbour in (steps - 1, steps + 1):  # never show the winner's own name next to it
            alternatives = [x for x in self.pools[i] if x != part]
            if strip[neighbour] == part and alternatives:
                strip[neighbour] = self.rng.choice(alternatives)
        reel = self.reels[i]
        reel.strip, reel.pos, reel.target, reel.landed = strip, 1, steps, False
        self._advance(i)
```

11.1 Animación sin bucle: temporizadores

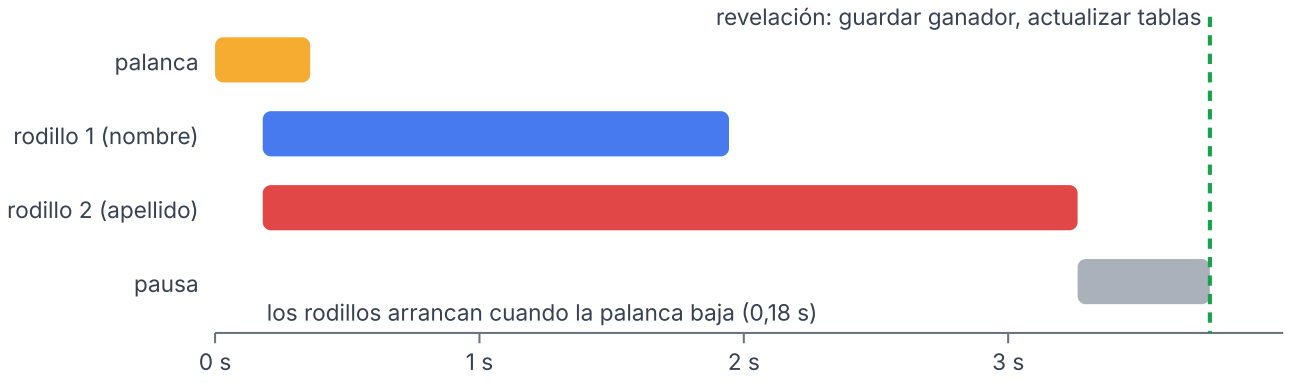
Podrías esperar un bucle `while` con `sleep` adentro. Eso congelaría toda la aplicación, porque Textual necesita seguir atendiendo teclas, el mouse y el dibujo. En cambio, cada paso hace un poco de trabajo y luego le pide a Textual: “llámame de nuevo en N segundos” con `set_timer` . Entre llamadas, Textual queda libre para hacer otras cosas.

src/raffle_cli/slot.py · SlotMachine._advance

```python
def _advance(self, i: int) -> None:
    reel = self.reels[i]
    reel.pos += 1
    if reel.pos >= reel.target:
        reel.landed = True
        self._pending -= 1
        self.refresh()
        if self._pending == 0 and self._on_done:
            self.set_timer(0.05 if self.fast else 0.5, self._on_done)
        return
    self.refresh()
    # slow down like a real machine: 40ms per row at first, ~320ms for the last rows
    delay = 0.02 if self.fast else 0.04 + 0.28 * (reel.pos / reel.target) ** 3
    self.set_timer(delay, lambda: self._advance(i))
```

La pausa crece con el cubo del progreso: `0.04 + 0.28 · (pos/steps)³`. Al principio las filas pasan cada 40 ms; cerca del final los intervalos se estiran a unos 0,3 s, igual que una máquina real que va frenando. Un cubo la mantiene rápida por mucho tiempo y luego frena fuerte. El rodillo derecho tiene 12 filas más que recorrer, así que aterriza después: el clásico suspenso de “se detiene el izquierdo, luego el derecho”.





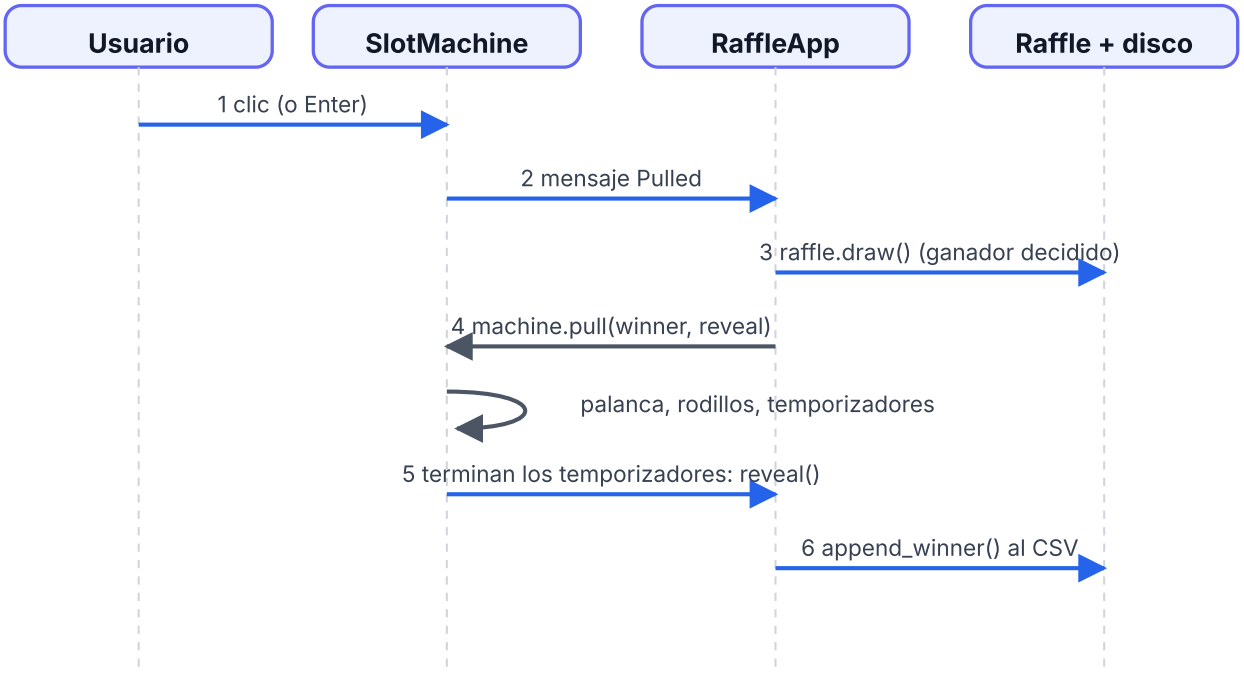
El tiempo total desde que se jala la palanca hasta la revelación es de unos **3.8 segundos** (calculado con la misma fórmula que usa el código). Con `--fast` las pausas bajan a 20 ms y los rodillos tienen 3 y 5 filas, lo que hace rápidas las pruebas automáticas.

11.2 La palanca y el mouse

src/raffle_cli/slot.py · SlotMachine.pull

```python
def pull(self, winner: Winner, on_done: Callable[[], None]) -> None:
    """Pull the lever; `on_done` runs once both reels have landed on `winner`."""
    self._on_done = on_done
    step = 0.02 if self.fast else 0.09
    self._set_lever(1)
    self.set_timer(step, lambda: self._set_lever(2))
    self.set_timer(2 * step, lambda: self._start_reels(winner))
    self.set_timer(3 * step, lambda: self._set_lever(1))
    self.set_timer(4 * step, lambda: self._set_lever(0))
```

La palanca tiene tres estados (arriba, a medias, jalada), cada uno simplemente una fila distinta para la bola roja. `pull()` los programa uno tras otro y luego pone a girar los rodillos cuando la palanca llega abajo. El clic funciona porque Textual llama al método `on_click` del widget con los clics del mouse; publicamos un mensaje `Pulled` y la aplicación (`on_slot_machine_pulled`) sortea. **Teclado y mouse ejecutan exactamente el mismo camino de código**, así que no hay nada extra que probar ni mantener.



Decisión técnica: dibujar la máquina como un solo widget de texto en vez de muchos widgets pequeños

Alternativa: armar la máquina con contenedores y etiquetas de Textual (una etiqueta por fila, un contenedor por rodillo). Sería más fácil de estilizar por separado, pero más difícil de animar sincronizadamente y de mantener la palanca alineada. Devolver un solo `Text` desde `render()` mantiene cada cuadro consistente y toda la máquina cabe en unas ~150 líneas. **Costo:** el arte con caracteres exige cuidado con los anchos: por eso cada línea se rellena a la misma longitud y el ancho de celda se limita a 8–12 caracteres (los nombres largos se cortan con "…").

12 Pruebas (tests)

Hay **43 pruebas automáticas**: 13 para el motor, 16 para archivos y configuración, 14 para comandos, fuegos artificiales y la aplicación Textual. Ejecútalas con `uv run pytest`. Una prueba es una función que prepara una situación, ejecuta el código y usa `assert` para afirmar lo que debe ser verdad. Si alguna afirmación es falsa, la prueba falla y te dice cuál.



La pirámide explica la estructura: muchas pruebas rápidas en la base para la lógica pura (milisegundos, sin archivos, sin pantalla), menos pruebas de más alto nivel arriba que ejercitan todo junto. Como separamos el motor de las interfaces en el capítulo 2, las reglas más importantes se pueden probar sin ninguna pantalla.

tests/test_raffle.py · tu regla “más premios que personas”

```python
def test_more_prizes_than_participants_are_shared_out_in_rounds():
    prizes = [Prize("a", 6), Prize("b", 4)]  # 10 prizes, 5 people
    winners = run_all(Raffle(PEOPLE, prizes, rng=random.Random(2)))
    assert len(winners) == 10
    assert set(Counter(w.full_name for w in winners).values()) == {2}  # fair: exactly two each
```

tests/test_raffle.py · reproducibilidad

```python
def test_same_seed_same_result():
    a = run_all(Raffle(PEOPLE, [Prize("a", 3)], rng=random.Random(42)))
    b = run_all(Raffle(PEOPLE, [Prize("a", 3)], rng=random.Random(42)))
    assert a == b
```

tests/test_cli.py · manejando la aplicación Textual real sin pantalla

```python
def test_tui_lever_click_draws_and_layout_is_responsive():
    from raffle_cli.slot import SlotMachine
    from raffle_cli.tui import RaffleApp

    people = [Participant("Ada", "Lovelace"), Participant("Guido", "van Rossum")]
    raffle = Raffle(people, [Prize("Mug", 2)], rng=random.Random(0))
    saved = []

    async def go():
        app_ = RaffleApp(raffle, saved.append, seed=0, fast=True, show_fireworks=False)
        async with app_.run_test(size=(140, 40)) as pilot:
            main = app_.query_one("#main")
            assert main.has_class("wide")
```

```
            await pilot.click(SlotMachine)  # mouse pulls the lever
            await pilot.pause(1.0)
            assert len(saved) == 1
            machine = app_.query_one(SlotMachine)
            assert [r.landed for r in machine.reels] == [True, True]
            assert [r.strip[r.pos] for r in machine.reels] == [saved[0].name, saved[0].lastname]
            assert machine.lever == 0
            await pilot.resize_terminal(90, 30)
            await pilot.pause()
            assert not main.has_class("wide")

    asyncio.run(go())
```

Decisión técnica: pytest + el Pilot de Textual, sin pytest-asyncio

Textual ofrece `app.run_test()`, que ejecuta la aplicación sin una terminal real y te da un **Pilot** para presionar teclas, hacer clic en widgets y cambiar el tamaño de la "terminal". El cuerpo de la prueba es `async`, así que lo envolvemos con `asyncio.run(go())` dentro de una función de prueba normal. **Alternativa:** el plugin `pytest-asyncio`, una dependencia más que explicar. **Costo:** unas cuantas llamadas a `pause()` para esperar los temporizadores; `--fast` las mantiene cortas.

Lo que las pruebas no pueden decirte

Todas estas pruebas pasan sin que nadie mire la pantalla. Las fuentes, cómo se ve la palanca en tu terminal y si los fuegos artificiales *se sienten* bien siguen necesitando tus propios ojos. Ejecuta el programa de verdad antes de un evento.

13 Decisiones técnicas en una tabla

Las mismas decisiones explicadas en los capítulos, como referencia rápida. *Costo* es a lo que renuncias.

Decisión	Alternativa	Por qué	Costo
Motor en un módulo puro (`raffle.py`)	lógica dentro del código de la interfaz	se puede probar y lo comparten ambas interfaces	un archivo más que aprender
Dataclasses inmutables	dicts / Pydantic	a prueba de errores de dedo, comparables, inmutables	sin conversión automática
Premios como lista de unidades	contadores por premio	menos casos especiales	la lista crece con el total de premios (diminuto)
Sorteo por la mínima `(wins, holds_item)`	azar con pregunta de cambio	justo, explicable, nunca entra en bucle	se eliminó el cambio de premio
Personas identificadas por posición	por nombre	se permiten nombres duplicados	necesita una advertencia de duplicados
Semilla visible, `Random(seed)`	`SystemRandom`	auditable, reproducible	no es secreta (bien para regalos)
Generador aleatorio cosmético aparte	reutilizar el del sorteo	la animación no puede cambiar a los ganadores	dos generadores que recordar
Decidir primero, guardar al revelar	guardar al sortear	el archivo no adelanta la revelación	salir a medio giro vuelve a sortear
Agregar ganadores uno a uno	escribir todo al final	a prueba de caídas	muchas escrituras pequeñas (diminuto)
Validar listando todos los problemas	fallar en el primer error	arreglar todo de una vez	un poco más de código
Nunca reparar automáticamente los archivos del usuario	ofrecer valores por defecto ante un error	no puede sobrescribir datos reales	el usuario arregla a mano
Rutas relativas al archivo de configuración	relativas a la carpeta del terminal	funciona desde cualquier lugar	algo sorprendente para algunos
Pydantic solo para la configuración	revisiones manuales de dict	errores claros con poco código	una dependencia
Callback de Typer con `invoke_without_command`	subcomando `run` explícito	mantiene el simple `raffle-cli`	las opciones van antes del subcomando
Textual como extra opcional, importación diferida	siempre instalado	instalación liviana por defecto	un paso extra de instalación
Simulación de fuegos artificiales propia	curses / biblioteca	portable, diminuta, educativa	mantenemos la física nosotros
Una simulación, dos formas de dibujarla	dos implementaciones	sin lógica duplicada	el dibujo solo devuelve `Text`
`Live(screen=True)` de Rich	imprimir cuadros y limpiar	sin parpadeo, la terminal se restaura	necesita una terminal real
Atajo con prioridad para Enter	hacer las tablas no enfocables	una línea, la tecla siempre funciona	tapa el Enter del widget
Una clase CSS cambia el diseño a las 120 columnas	dos pantallas separadas	un solo diseño, poco código	punto de corte fijo
Tragamonedas = un solo widget propio	muchos widgets pequeños	los cuadros se mantienen consistentes	cuidado con los anchos del texto
Nombres en texto plano, sin fuente ASCII-art	letras grandes con figlet	nombres largos y tildes rompen el arte	nombres más pequeños

Temporizadores en vez de bucles con `sleep`	animación bloqueante	la interfaz sigue respondiendo	callbacks más difíciles de leer
`asyncio.run` en las pruebas	pytest-asyncio	sin plugin extra	un poco de código repetitivo
Python ≥ 3.10, uv + archivo de bloqueo	solo versiones nuevas / pip freeze	amplio soporte, instalaciones reproducibles	la sintaxis antigua limita

14 Lo que salió mal (y lo que enseñó)

Los proyectos reales no avanzan en línea recta. Estos son los desvíos reales, porque las historias de depuración enseñan más que el código pulido.

Qué pasó	Causa	Lección
Presionar Enter en la aplicación Textual no hacía nada	La tabla enfocada manejó Enter primero	Los eventos van primero al widget enfocado; usa `priority=True`. Las pruebas que simulan teclas lo encuentran en segundos.
El mensaje "instala el extra tui" se cortó a `pip install 'raffle-cli'`	Rich leyó `[tui]` como marcado y se lo tragó	Escapa todo lo visible para el usuario que pueda contener corchetes (`rich.markup.escape`).
`uvx ... --tui` falló en una máquina nueva	`uvx` no instala extras opcionales si no se piden	Documenta el comando exacto (`--with textual`) y haz que el error lo diga.
Misma semilla, ganadores distintos en las dos interfaces	La animación consumía números del generador del sorteo	Separa la aleatoriedad que debe ser reproducible de la que es decoración.
Los nombres grandes en ASCII-art se veían impresionantes y luego se descartaron	Nombres largos, tildes y letras no latinas se desbordan o se dibujan en blanco	Las funciones que necesitan manejar anchos para cualquier entrada posible son caras. El texto plano más simple gana en una terminal.
La primera idea del giro mostraba un nombre a la vez	Queríamos "un giro literal"	Pasar a rodillos (una lista y un puntero) hizo el efecto más convincente *y* más simple que el código anterior.

15 Ejercicios

Aquí es donde recuperas el código. Están ordenados del más fácil al más difícil. Para cada uno, escribe primero una prueba (o al menos ejecuta el programa y mira).

#	Ejercicio	Dónde mirar	Pista
1	Cambia los colores de los fuegos artificiales y haz que duren más	`fireworks.py` `PALETTE`, segundos en `ui.py`	Ejecuta `raffle-cli fireworks --seconds 20` mientras ajustas.
2	Agrega un nuevo valor de `--order`, `alpha` (premios de la A a la Z)	`raffle.py` `ORDERS`, `order_prizes`	Un `elif` y una prueba. `sort(key=lambda p: p.item.lower())`.
3	Imprime una frase al final: "N ganadores, M premios"	`ui.show_results`	Tienes `raffle.winners` y `raffle.remaining`.
4	Agrega una columna opcional `email` a los participantes que el sorteo ignore	`files.py`, `raffle.py`	La revisión del encabezado es estricta: decide cómo permitir columnas extra y prueba ambos casos.
5	Agrega una opción `--exclude "Nombre Apellido"` para personas presentes pero no elegibles (organizadores)	`cli.py` `build_raffle`	Filtra la lista de participantes antes de crear el `Raffle`.
6	Exporta un archivo de resultados en Markdown junto a `winners.csv`	nueva función en `files.py`	Llámala después del último premio en `main`.
7	Agrega un tercer rodillo que muestre el premio en la tragamonedas	`slot.py`	La lista de rodillos está hecha para 2; generalízala. Ojo con el ancho total y el ancho del dock en el CSS (52).
8	Agrega una columna `weight` para que algunas personas tengan más oportunidades	`raffle.py` `draw`	Piensa primero en la justicia: ¿cómo interactúa con "todos ganan una vez antes de que alguien gane dos"? Escribe esa regla antes de programar.

16 Glosario y cómo ejecutar todo

Ejecútalo

Sin instalar (macOS / Linux):

```bash / zsh
R="git+https://github.com/andoresuperesu/raffle-cli"
uvx --from $R raffle-cli setup
uvx --from $R --with textual raffle-cli --tui
```

PowerShell:

```PowerShell
$R = "git+https://github.com/andoresuperesu/raffle-cli"
uvx --from $R raffle-cli setup
uvx --from $R --with textual raffle-cli --tui
```

Desde un clon, para desarrollar:

```cualquier shell
git clone https://github.com/andoresuperesu/raffle-cli
cd raffle-cli
uv sync
uv run pytest
uv run raffle-cli --config examples/config.yaml --tui
```

Límites conocidos

- Desarrollado y probado en Linux. En Windows usa Windows Terminal; los caracteres de dibujo de cajas y la entrada del mouse no están verificados allí.
- Las pruebas demuestran el comportamiento, no la apariencia: revisa visualmente la tragamonedas y los fuegos artificiales antes de un evento.
- La semilla de 32 bits hace los sorteos reproducibles, no secretos (ver capítulo 7).

Glosario

Callback	Una función que le entregas a otro código para que te llame de vuelta más tarde (`on_winner` , `reveal`).
CLI	Interfaz de línea de comandos: un programa que controlas escribiendo un comando.
Dataclass	Una clase pensada para guardar datos; Python genera los métodos aburridos.
Punto de entrada (entry point)	La función que llama un comando como `raffle-cli` ; se declara en `pyproject.toml` .
Extra	Dependencias opcionales que se instalan con `paquete[nombre]` .
Frozen (inmutable)	Un objeto que no se puede modificar después de creado.

Pilot	El conductor de pruebas de Textual que presiona teclas y hace clics en una aplicación sin pantalla.
Pseudoaleatorio	Parece aleatorio pero se calcula a partir de una semilla, así que se puede repetir.
Función / módulo puro	Solo calcula a partir de sus entradas: sin imprimir, sin archivos, sin reloj.
Semilla (seed)	El número inicial de un generador pseudoaleatorio.
Separación de responsabilidades	Dividir un programa para que cada parte tenga un solo trabajo.
Tic (tick)	Un paso de una simulación o animación.
TUI	Interfaz de usuario de texto: una aplicación a pantalla completa dibujada con caracteres.
Entorno virtual	Una carpeta privada de paquetes instalados para un proyecto (`.venv`).
Widget	Una pieza reutilizable de una interfaz de usuario (una tabla, una etiqueta, la tragamonedas).

Dónde seguir leyendo

- Typer: typer.tiangolo.com · Rich: rich.readthedocs.io · Textual: textual.textualize.io
- Pydantic: docs.pydantic.dev · pytest: docs.pytest.org · uv: docs.astral.sh/uv
- El tutorial de Python sobre clases, excepciones y módulos: docs.python.org/es/3/tutorial